

Complexity-conditioned routing for daily rainfall–runoff prediction

What `compare_learned_morse_hydrology_consolidation.py` does

Ensemble Review

The goal

Question. Can a light and a high-capacity runoff model be combined more robustly by routing each input window according to its local complexity?

The entry point is an *isolated boundary-consolidation experiment*. It keeps the standard learned-Morse workflow intact and varies one extra loss weight:

$$\lambda_i = \text{-consolidation-weight}.$$

- ▶ Forecast target: next-day specific discharge.
- ▶ Principle: use a soft, interpretable choice between a simple and a complex expert.
- ▶ Important: the accompanying paper is a methods draft; it reports **no results**.

Inputs become leakage-safe hydrology windows

The script loads one basin from either CAMELS-CH or a prepared Ukrainian daily CSV, then calls `make_hydrology_data`.

Stage	What happens	Why it matters
Chronological split	Default 60 / 20 / 20 train-validation-test	Prevents future-data leakage
Features	Forcing, prior flow, seasonal, optional land-cover features	Captures forcing and state
Windowing	Previous 30 days by default	Supplies recent history
Scaling	Fit training data; reuse unchanged	Prevents normalization leakage
Long score context	TDA 128 days; Lyapunov 365 days	Past-only dynamical scoring

A soft mixture of two experts

```
past forcing / flow window x_t
|
+-- simple expert f_s    (RBF or Fourier)
+-- complex expert f_c   (MLP or PINNMamba)
`-- complexity router --> complex weight w_t in [0, 1]
                        |
                        (1 - w_t) f_s(x_t) + w_t f_c(x_t)
                        |
                        softplus --> non-negative discharge y_hat_t
```

$$\hat{y}_t = \text{softplus}((1 - w_t)f_s(x_t) + w_tf_c(x_t)).$$

The comparison also includes a **single complex expert**, so any apparent routing benefit is judged against a capacity-matched non-routed baseline.

How the router decides when complexity is high

For score-based routes, the score threshold is fitted only on the training split:

$$w_t = \sigma\left(\frac{s_t - \tau}{\gamma \text{IQR}(s)}\right).$$

- ▶ τ : training-score percentile (`--complexity-percentile`, default 80).
- ▶ γ : transition temperature relative to the training IQR (`--gate-temperature`, default 0.15).
- ▶ The logistic gate remains soft; it does not force a hard regime label.

Selectable routes:

Route	Score source
morse	Gradient norm of a hand-specified hydrological potential
learned_morse	Gradient norm of a pretrained scalar potential
learned	Jointly trained neural gate; interpretability baseline
tda / lyapunov	Optional external dynamical-complexity scores

Learned-Morse: learn a score tied to expert utility

learned_morse is calibrated before the final routed model is trained.

1. **Forward-chain the training split.** Pilot simple and complex models are trained on each prefix, then scored on its next fold.
2. **Create out-of-sample utility labels.** Utility is the complex model's advantage in squared error plus weighted physics-residual advantage.
3. **Learn the potential.** A small network maps window summaries—current value, mean, variation, change, and recent means—to $V(x)$; pairwise ranking makes $\|\nabla V(x)\|$ increase with utility.
4. **Freeze and reuse it.** The final ensemble routes with $s(x) = \|\nabla V(x)\|$, rather than updating the potential jointly.

Calibration and transfer reports test score–utility alignment with Spearman correlation, top-20% precision, and positive utility at high versus low scores.

Boundary consolidation: agree where the gate is uncertain

The usual data, physics, and router-regularization losses are retained. This experiment adds:

$$L_{\text{interface}} = \frac{1}{N} \sum_t 4w_t(1 - w_t) [f_s(x_t) - f_c(x_t)]^2.$$

Gate behavior	Consolidation effect
$w_t \approx 0$ or 1	Nearly zero: experts can specialize
$w_t = 0.5$	Maximum: experts are encouraged to agree

The total objective is

$$L = L_{\text{data}} + \lambda_p L_{\text{physics}} + \lambda_i L_{\text{interface}} + \lambda_r L_{\text{routing}} + \lambda_c \bar{w}.$$

The key ablation is identical except `--consolidation-weight 0`; this isolates whether

How the comparison is run and judged

For each selected model and seed, the script trains under two input conditions and tests under two input conditions:

	Clean inference	Noisy inference
Clean training	reference	inference robustness
Noisy training	cost of training noise	matched-noise robustness

Noise is Gaussian in standardized units and is limited to dynamic observations: precipitation, temperature, PET, and previous discharge.

Reported per run: validation/test NSE, test KGE, RMSE (mm/day), reservoir physics error, parameter count, mean complex weight, mean complexity score, valid-score fraction, and consolidation weight.

Reproducible outputs and responsible interpretation

Example invocation:

```
python -m examples.compare_learned_morse_hydrology_consolidation \
  --source camels_ch --basin 2011 --simple rbf --complex mlp \
  --epochs 750 --consolidation-weight 0.05 \
  --training-noise 0.1 --inference-noise 0.1
```

The run writes a complete JSON configuration, comparison CSV, grouped robustness plot, and—when `learned_morse` is selected—calibration CSV, transfer CSV, and frozen potential weights.

Interpret results conservatively: aggregate paired comparisons across basins and seeds; examine full robustness grids and routing diagnostics; do not infer a universal physical regime or superiority from one basin, seed, or configuration.